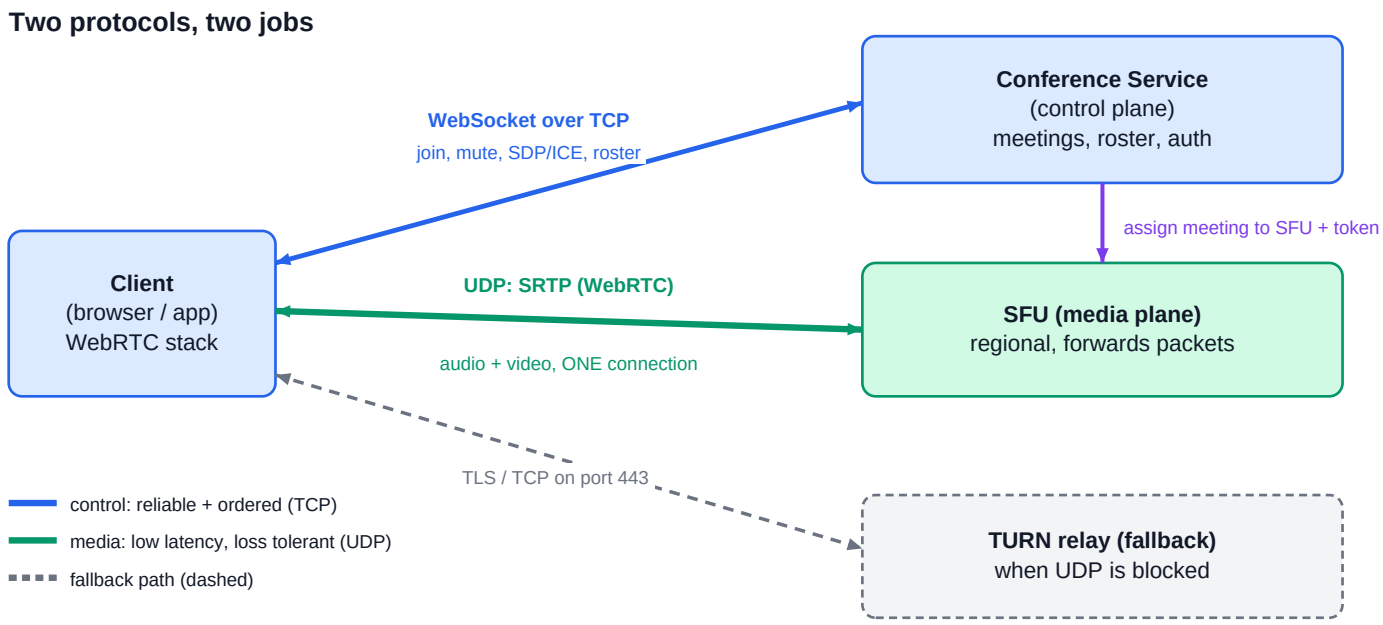
Video Conferencing (Zoom)

Interview cheat sheet: corrected design, HLD, and the LLD areas most likely to be probed

1. Protocol decision

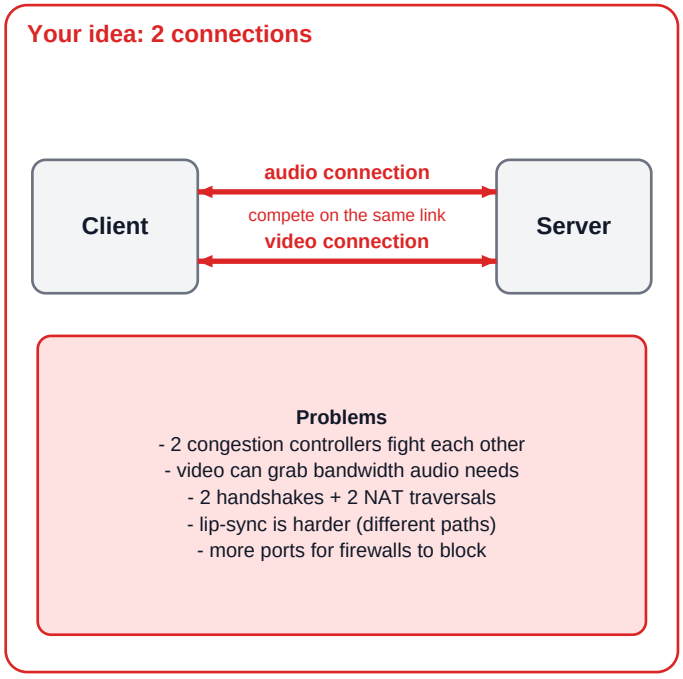
WebSocket runs on top of TCP, so the real choice is **TCP-based vs UDP**. The answer is both, for different jobs: UDP for media, WebSocket for signaling.

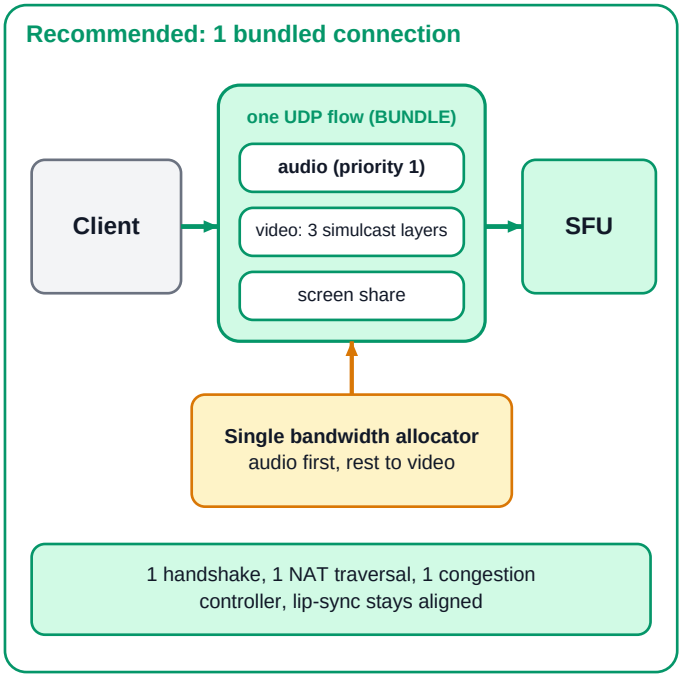


Traffic	Protocol	Why
Audio and video	UDP (SRTP via WebRTC)	A late packet is a lost packet. TCP retransmits cause head-of-line blocking: a freeze, then a burst of stale frames.
Signaling, roster, chat	WebSocket (TCP)	Small messages that must be reliable and ordered; tens of ms of delay is fine.
UDP blocked by firewall	TURN over TLS/TCP on 443	Works through strict corporate firewalls, at the cost of higher latency.

2. One connection or two?

The goal is right: protect audio, sacrifice video under bad networks. The mechanism should be **one bundled connection with an allocator**, not two competing connections.



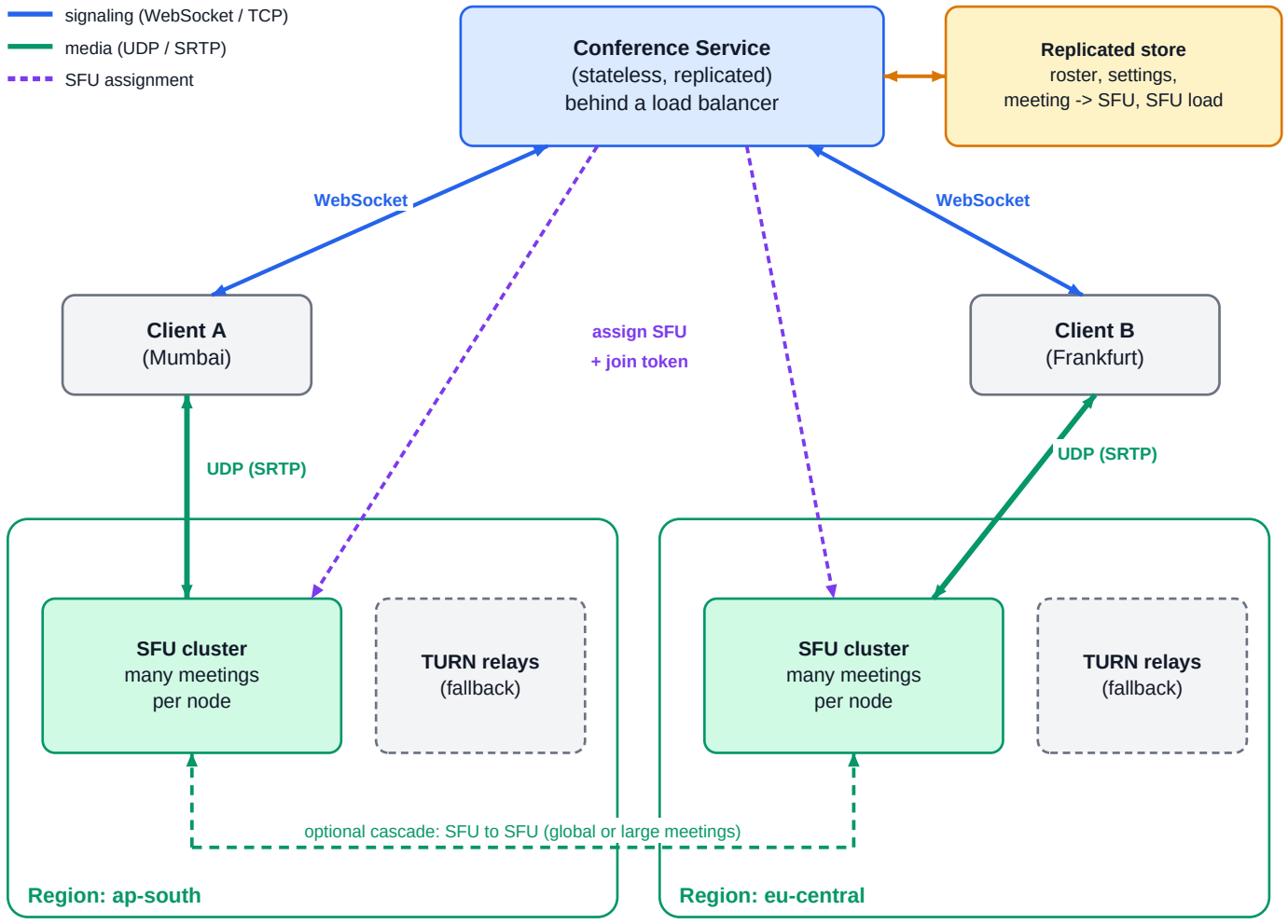


How video is dialed down without a second connection

- Lower the resolution (720p to 360p to 180p) or the frame rate (30 to 15 fps).
- Lower the encoder bitrate, or switch the simulcast layer the SFU forwards (no re-encoding).
- Turn video off as the last resort and keep audio, which needs only ~30-50 kbps.
- Audio is further protected with Opus in-band FEC and redundant audio packets.

3. High-level design

Split the system into a **control plane** (conference service) and a **media plane** (regional SFUs). They scale and fail differently.



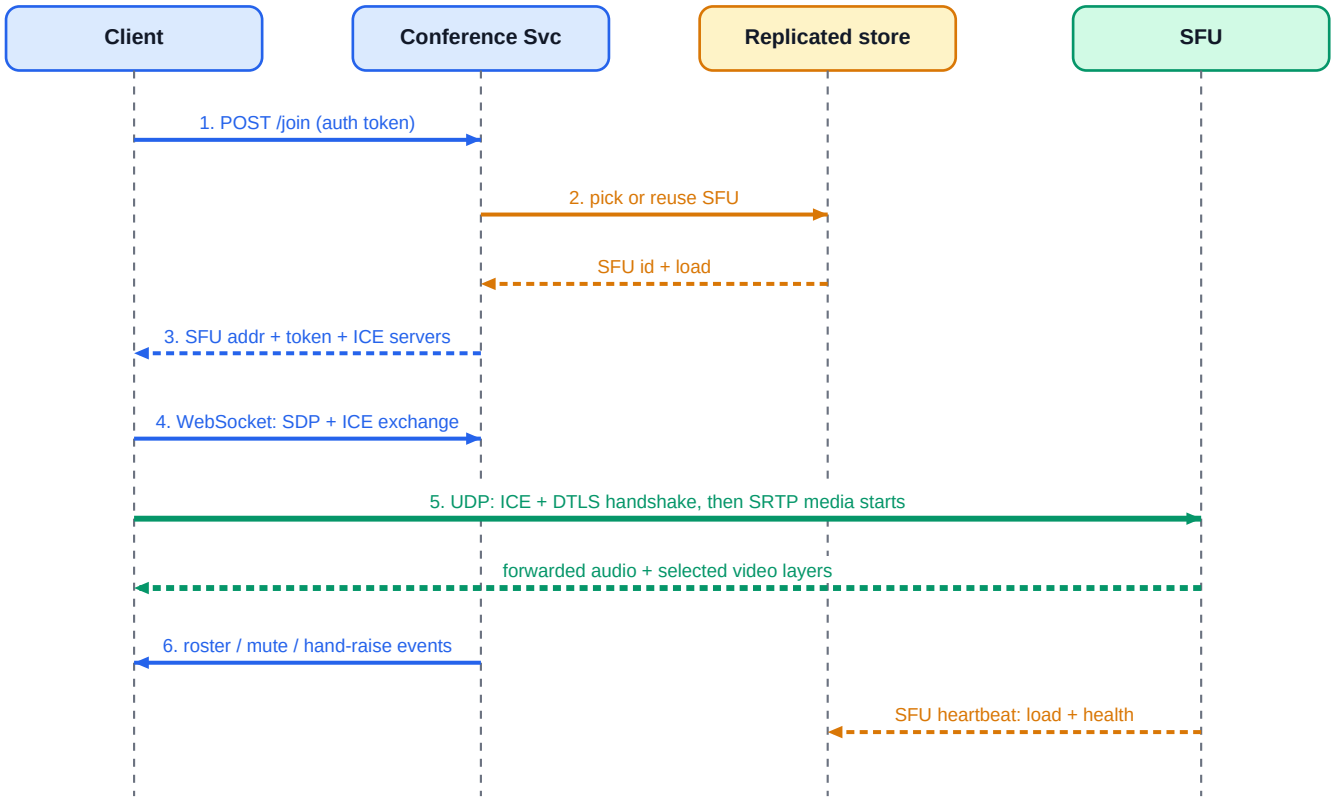
Why an SFU and not a pub/sub broker, mesh, or MCU

Option	Server CPU	Client uplink	Verdict
Mesh (peer to peer)	None	N-1 streams	Breaks down beyond ~4 people; fine for 1:1
MCU (server mixes)	Very high	1 stream	Decode + re-encode per participant; added latency
SFU (server forwards)	Low	1 stream (simulcast)	Chosen: bandwidth-heavy but cheap, per-receiver quality
Pub/sub broker	Low	1 stream	Built for durability and ordering, too slow for 20 ms media

* Selective Forwarding Unit

4. Join flow (control plane to media plane)

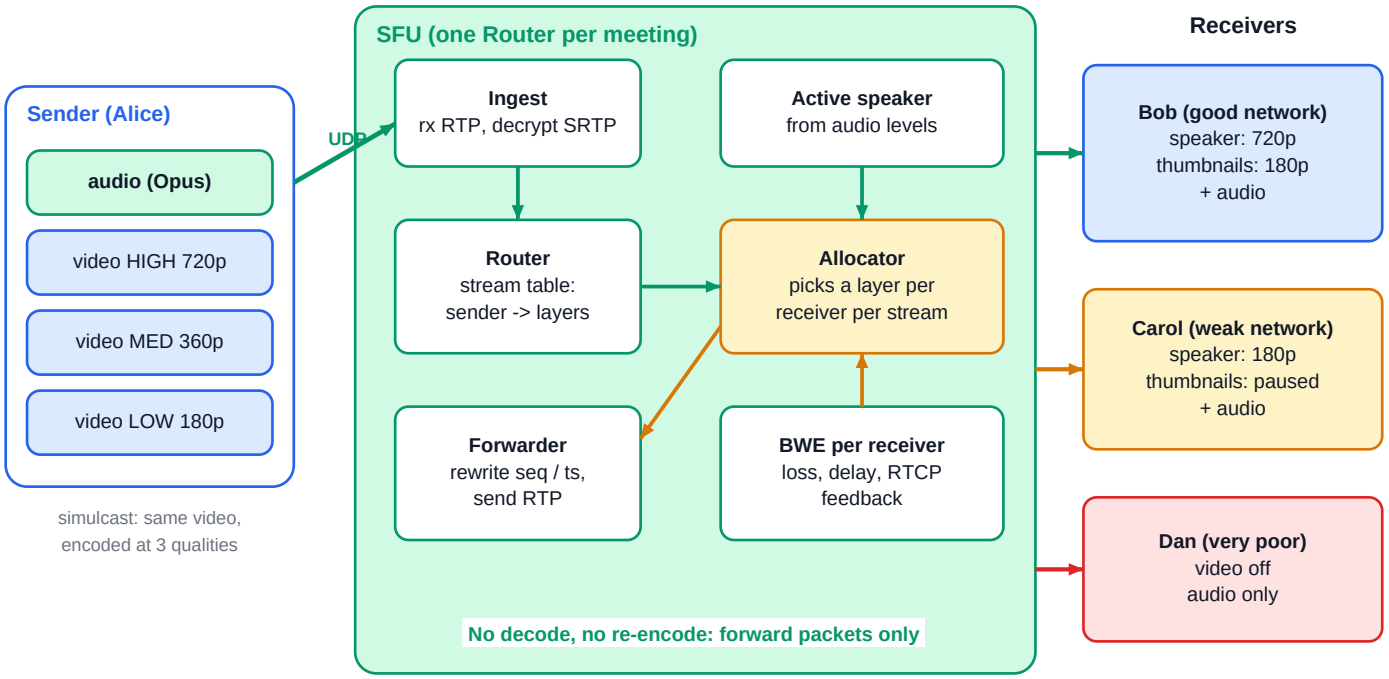
The conference service never touches media. It authenticates, assigns the meeting to the nearest SFU, and hands the client what it needs to connect.



- **Why separate signaling from media:** signaling is small, ordered and reliable (WebSocket); media is latency-critical (UDP). Each scales on its own metric.
- **Why media survives a signaling outage:** once the UDP session exists, the SFU keeps forwarding even if the WebSocket to the conference service is down.
- **Regional placement:** the SFU is chosen in the region closest to the participants, so a user in Gaya is not sent to a server in Virginia.

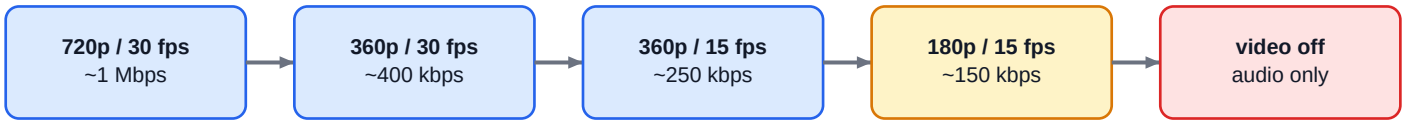
5. LLD A: inside the SFU (most likely deep dive)

Each client sends **one** upstream with multiple quality layers. The SFU forwards packets without decoding and picks, per receiver, which layer of which stream to send.



Degradation ladder (approx. bitrates)

bandwidth drops: step DOWN fast | bandwidth recovers: step UP slowly (hysteresis)



Audio (Opus, ~30-50 kbps) is always kept, protected by FEC / redundancy. Video degrades first.

6. LLD B: conference service data model and key logic

Entity	Key fields	Lives in
Meeting	meetingId (PK), hostId, status, region, **sfuId**, settings (passcode, waiting room)	Replicated store
Participant	(meetingId, participantId), userId, role, muted, joinedAt, connectionState	Replicated store
SfuNode	sfuId, region, address, participants, bandwidthUsed, health, lastHeartbeat	Replicated store (fed by SFU heartbeats)
Subscription	receiverId, senderId, currentLayer, paused	SFU memory only (rebuilt on reconnect)

Meeting placement (join path)

```
join(meetingId, userId):
  meeting = store.get(meetingId)
  if meeting.sfuId is None:
      sfu = pick_sfu(nearest_region(client), least_loaded, headroom=20%)
      ok  = store.compare_and_set(meetingId, sfuId=None, new=sfu.id)   # atomic
      if not ok: sfu = store.get(meetingId).sfu       # another joiner won the race
  token = sign(meetingId, userId, sfu.id, ttl=5min)
  return { sfu.address, token, ice_servers }
```

- **Probe:** two people join an empty meeting at the same moment. Without an atomic compare-and-set, the meeting lands on two SFUs and the call splits in two.

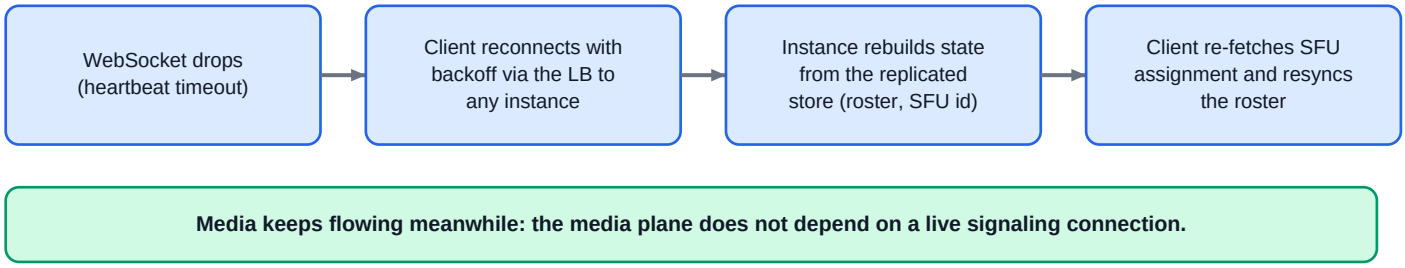
Bandwidth allocator (runs on every bandwidth update for one receiver)

```
on_bwe_update(receiver):
  budget  = bwe(receiver) * 0.85              # keep headroom
  budget -= audio_cost(top_3_loudest)         # audio always comes first
  for s in by_priority(subscriptions):        # screen share, active speaker, thumbnails
      layer = best_layer(s, fits=budget)      # highest simulcast layer under the budget
      if layer is None: pause(s)
      else:             forward(s, layer); budget -= layer.bitrate
  # switch layers only on a keyframe (send PLI); step down fast, step up slowly
```

7. Failure handling (the question to expect)

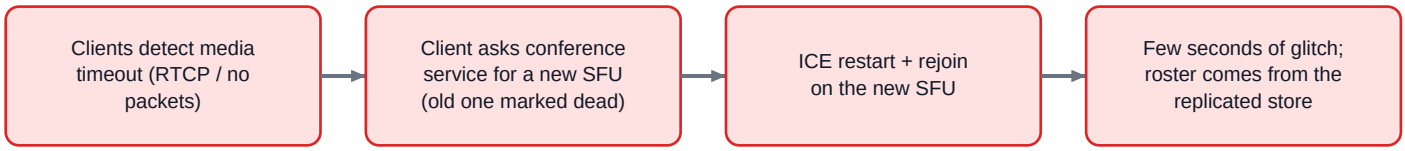
Which state is needed to recover? The roster, settings and SFU assignment are small and live in a replicated store, so any conference-service instance can take over.

A) Conference service instance crashes



Media keeps flowing meanwhile: the media plane does not depend on a live signaling connection.

B) SFU crashes mid-meeting



30-second pitch

I would split signaling from media. Signaling goes over WebSocket since it needs reliability and ordering. Media goes over UDP using WebRTC, because for live audio and video a late packet is as bad as a lost one, and TCP retransmission causes head-of-line blocking. I would use a single bundled connection per client rather than separate audio and video connections, so one congestion controller can protect audio and degrade video first through simulcast layers. Media is forwarded by regional SFUs, and a stateless conference service backed by a replicated store assigns each meeting to the nearest one.

Likely probes

- What happens if the conference service crashes mid-meeting? (Section 7, case A)
- Why not two connections, or a pub/sub broker, for media? (Sections 2 and 3)
- How do you handle one participant on a very poor network without hurting the others? (Per-receiver allocator, Section 5)
- Two people join an empty meeting at once; what stops a split? (Atomic assignment, Section 6)